



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 1

Тема занятия: 1) Создание пользовательских функций
2) Создание пользовательских функций. Язык pl/pgSQL
3) Создание пользовательских функций. Работа с датой и временем

Выполнил студент 2 курса группы
ЭФБО-04-24 Валекжанин В.С.

Проверил доцент Бочаров М.И.

Москва
2025 г.

Практическая работа 1

Вывод \df

List of functions				
Schema	Name	Argument data types	Result data type	Type
public	calculate_age_valekzhanin	integer		func
public	calculate_monthly_payment_valekzhanin	numeric		func
public	categorize_order_valekzhanin	text		func
public	convert_currency_valekzhanin	numeric		func
public	generate_order_numbers_valekzhanin	TABLE(order_number text)		func
public	get_properties_below_average_price_valekzhanin	TABLE(address character varying, district_name character varying, room_count smallint)		func
public	get_realtor_salary_valekzhanin	numeric		func
public	get_sales_statistics_valekzhanin	TABLE(property_type character varying, sales_count bigint, percentage numeric, total_amount numeric)		func
public	transfer_money_valekzhanin	boolean		func
public		from_account_id integer, to_account_id integer, amount numeric		func

(9 rows)

Практическая работа 2.1

Задание 1

Задание 1: Функция для расчета возраста

Напишите функцию calculate_age, которая принимает дату рождения и возвращает возраст в годах на текущую дату.

Требования:

- Имя функции: calculate_age
- Параметр: birth_date типа DATE
- Возвращает: INTEGER
- Используйте функцию AGE() и EXTRACT(YEAR FROM ...)
- Обработайте случай NULL значения

```
1 CREATE OR REPLACE FUNCTION calculate_age_valekzhanin
2 (birth_date DATE)
3 RETURNS INTEGER
4 RETURNS NULL ON NULL INPUT
5 AS $$
6     SELECT EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date))
7     :: INTEGER;
8 $$ LANGUAGE SQL;
```

```
test_calculate_age.sql test_categorize_order
real_estate_db=# \i 2.1/test/test_calculate_age.sql
age
----
 19
(1 row)

age
----
(1 row)
```

Задание 2

Задание 2: Функция категоризации заказа

Напишите функцию `categorize_order`, которая определяет категорию заказа на основе его суммы.

Требования:

- Имя функции: `categorize_order`
- Параметр: `order_amount` типа `DECIMAL(10,2)`
- Возвращает: `TEXT`
- Логика:
 - `< 1000`: 'МАЛЫЙ'
 - `1000-5000`: 'СРЕДНИЙ'
 - `5000-20000`: 'КРУПНЫЙ'
 - `20000`: 'ОСОБО КРУПНЫЙ'
- Добавьте проверку на отрицательные значения

```
1 CREATE OR REPLACE FUNCTION
  generate_order_numbers_valekzhanin(count INTEGER)
2 RETURNS TABLE(order_number TEXT)
3 RETURNS NULL ON NULL INPUT
4 AS $$
5     SELECT 'ORD-' || TO_CHAR(NOW(), 'YYYYMM') || LPAD(FLOOR
6       (RANDOM() * 1000000)::TEXT, 6, '0')
7     FROM generate_series(1, count);
8 $$ LANGUAGE SQL;
```

```
real_estate_db=# \i 2.1/test/test_categorize_order.sql
result
-----
ОТРИЦАТЕЛЬНОЕ ЧИСЛО
(1 row)

result
-----
МАЛЫЙ
(1 row)

result
-----
СРЕДНИЙ
(1 row)

result
-----
КРУПНЫЙ
(1 row)

result
-----
ОСОБО КРУПНЫЙ
(1 row)

result
-----
(1 row)
```

Задание 3

Задание 3: Функция генерации номеров заказов

Напишите функцию `generate_order_numbers`, которая генерирует указанное количество тестовых номеров заказов.

Требования:

- Имя функции: `generate_order_numbers`
- Параметр: `count` типа `INTEGER` (количество номеров для генерации)
- Возвращает: `TABLE(order_number TEXT)`
- Формат номера: 'ORD-' + год + месяц + 6 случайных цифр
- Используйте цикл `FOR` или `WHILE`
- Используйте `RETURN NEXT` для возврата набора строк

```
1 CREATE OR REPLACE FUNCTION categorize_order_valekzhanin
  (order_amount DECIMAL(10,2))
2 RETURNS TEXT
3 RETURNS NULL ON NULL INPUT
4 AS $$
5 BEGIN
6     IF order_amount < 0 THEN
7         RETURN 'ОТРИЦАТЕЛЬНОЕ ЧИСЛО';
8     ELSIF order_amount < 1000 THEN
9         RETURN 'МАЛЫЙ';
10    ELSIF order_amount < 5000 THEN
11        RETURN 'СРЕДНИЙ';
12    ELSIF order_amount < 20000 THEN
13        RETURN 'КРУПНЫЙ';
14    ELSE
15        RETURN 'ОСОБО КРУПНЫЙ';
16    END IF;
17 END;
18 $$ LANGUAGE plpgsql;
```

```
[real_estate_db=# \i 2.1/test/test_generate_order_numbers.sql
result
-----
ORD-202603510049
ORD-202603515727
ORD-202603668801
(3 rows)

result
-----
(0 rows)

result
-----
(0 rows)

result
-----
(0 rows)
```

Задание 4

Функция перевода средств между счетами

Напишите функцию `transfer_money`, которая выполняет перевод денег между двумя счетами.

Требования:

- Имя функции: `transfer_money`
- Параметры:
 - `from_account_id` INTEGER
 - `to_account_id` INTEGER
 - `amount` DECIMAL(10,2)
- Возвращает: BOOLEAN (успех/неудача)
- Логика:
 1. Проверить существование обоих счетов
 2. Проверить достаточность средств на отправителе
 3. Выполнить перевод в транзакции
 4. Добавить запись в таблицу `transactions`
 5. Обработать возможные ошибки с помощью `EXCEPTION`

```
CREATE TABLE accounts (  
    id SERIAL PRIMARY KEY,  
    balance DECIMAL(10,2) NOT NULL DEFAULT 0  
);
```

```
CREATE TABLE transactions (  
    id SERIAL PRIMARY KEY,  
    from_account INTEGER,  
    to_account INTEGER,  
    amount DECIMAL(10,2),  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

```

1  CREATE OR REPLACE FUNCTION transfer_money_valekzhanin(
2      from_account_id INTEGER,
3      to_account_id INTEGER,
4      amount DECIMAL(10,2)
5  )
6  RETURNS BOOLEAN
7  AS $$
8  DECLARE
9      sender_record accounts%ROWTYPE;
10     receiver_record accounts%ROWTYPE;
11 BEGIN
12     IF from_account_id IS NULL OR to_account_id IS NULL OR
13     amount IS NULL THEN
14         RETURN FALSE;
15     END IF;
16
17     IF amount ≤ 0 THEN
18         RETURN FALSE;
19     END IF;
20
21     -- Проверка существования счетов и получение их данных
22     SELECT * INTO sender_record FROM accounts WHERE id =
23     from_account_id;
24     IF NOT FOUND THEN
25         RETURN FALSE;
26     END IF;
27
28     SELECT * INTO receiver_record FROM accounts WHERE id =
29     to_account_id;
30     IF NOT FOUND THEN
31         RETURN FALSE;
32     END IF;
33
34     -- Проверка достаточности средств
35     IF sender_record.balance < amount THEN
36         RETURN FALSE;
37     END IF;
38
39     -- Выполнение перевода (списание и зачисление)
40     UPDATE accounts SET balance = balance - amount WHERE id
41     = from_account_id;
42     UPDATE accounts SET balance = balance + amount WHERE id
43     = to_account_id;
44
45     -- Логирование транзакции
46     INSERT INTO transactions (from_account, to_account,
47     amount)
48     VALUES (from_account_id, to_account_id, amount);
49
50     RETURN TRUE;
51
52 EXCEPTION WHEN OTHERS THEN
53     RETURN FALSE;
54 END;
55 $$ LANGUAGE plpgsql;

```

```

test_calculate_age.sql      test_categorize_order.sql
real_estate_db=# \i 2.1/test/test_transfer_money.sql
result
-----
t
(1 row)

result
-----
f
(1 row)

result
-----
f
(1 row)

result
-----
f
(1 row)

result
-----
f
(1 row)

result
-----
f
(1 row)

result
-----
f
(1 row)

```

Практическая работа 2.2

Задание 1

1. Конвертирует стоимость объекта недвижимости в другую валюту.

Вх. параметр	Вых. параметр
курс в рублях, код объекта недвижимости	стоимость в другой валюте

```

1  CREATE OR REPLACE FUNCTION convert_currency_valekzhanin(
2      p_exchange_rate DECIMAL(10,2),
3      p_property_id INTEGER
4  )
5  RETURNS DECIMAL(10,2)
6  AS $$
7  DECLARE
8      v_price INTEGER;
9      v_converted_price DECIMAL(10,2);
10 BEGIN
11     -- Получаем цену объекта недвижимости
12     SELECT price INTO v_price
13     FROM property
14     WHERE id = p_property_id;
15
16     -- Проверяем, найден ли объект
17     IF v_price IS NULL THEN
18         RAISE EXCEPTION 'Объект недвижимости с id % не
19         найден', p_property_id;
20     END IF;
21
22     -- Конвертируем цену
23     v_converted_price := v_price / p_exchange_rate;
24
25     RETURN v_converted_price;
26 END;
27 $$ LANGUAGE plpgsql;

```

Задание 2

2. Рассчитывает заработную плату риэлтора за месяц по формуле: $N \cdot S + R$, где N – общая сумма проданных объектов недвижимости в месяц (подсчет осуществляется автоматически по данным таблицы «Продажи»), S – коэффициент, R – премия.

Вх. параметр	Вых. параметр
S, R, месяц, год, фамилия риэлтора	размер заработной платы


```

1 CREATE OR REPLACE FUNCTION get_realtor_salary_valekzhanin(
2     p_coefficient DECIMAL(10,2),
3     p_bonus DECIMAL(10,2),
4     p_month INTEGER,
5     p_year INTEGER,
6     p_realtor_last_name VARCHAR(255)
7 )
8 RETURNS DECIMAL(10,2)
9 AS $$
10 DECLARE
11     v_total_sales DECIMAL(10,2);
12     v_salary DECIMAL(10,2);
13     v_realtor_id INTEGER;
14 BEGIN
15     -- Получаем ID риэлтора по фамилии
16     SELECT id INTO v_realtor_id
17     FROM realtor
18     WHERE last_name = p_realtor_last_name;
19
20     -- Проверяем, найден ли риэлтор
21     IF v_realtor_id IS NULL THEN
22         RAISE EXCEPTION 'Риэлтор с фамилией % не найден',
23             p_realtor_last_name;
24     END IF;
25
26     -- Рассчитываем общую сумму проданных объектов за
27     -- указанный месяц
28     SELECT COALESCE(SUM(price), 0) INTO v_total_sales
29     FROM sale
30     WHERE realtor_id = v_realtor_id
31     AND EXTRACT(MONTH FROM sale_date) = p_month
32     AND EXTRACT(YEAR FROM sale_date) = p_year;
33
34     -- Рассчитываем зарплату по формуле N*S+R
35     v_salary := v_total_sales * p_coefficient + p_bonus;
36     RETURN v_salary;
37 END;
38 $$ LANGUAGE plpgsql;

```

```

real_estate_db=# \i 2.2/test/test_get_realtor_salary.sql
result
-----
10000.00
(1 row)

result
-----
0.00
(1 row)

result
-----
(1 row)

psql:2.2/test/test_get_realtor_salary.sql:11: ERROR: Риэлтор с фамилией <NULL> не найден
CONTEXT: PL/pgSQL function get_realtor_salary_valekzhanin(numeric,numeric,integer,integer,character varying) line 14 at RAISE
psql:2.2/test/test_get_realtor_salary.sql:14: ERROR: Риэлтор с фамилией Петров не найден
CONTEXT: PL/pgSQL function get_realtor_salary_valekzhanin(numeric,numeric,integer,integer,character varying) line 14 at RAISE

```

Задание 3

3. Рассчитывает сумму ежемесячного платежа по ипотеке. **Входные параметры:** код объекта недвижимости, процентная ставка, срок (в годах), первоначальный взнос.

$$\text{Аннуитетный платеж} = \text{Сумма кредита} * \frac{i * (1 + i)^n}{(1 + i)^n - 1}$$

где **n** — количество месяцев, в которые платится ипотека.

i — процентная ставка по займу в месяц.

Если у вас указана годовая ставка, нужно поделить ее на 12.

Т.е. допустим годовая ставка - 12%, тогда **i = 0.12/12 = 0.01**

```

1  CREATE OR REPLACE FUNCTION
   calculate_monthly_payment_valekzhanin(
2      p_property_id INTEGER,
3      p_interest_rate DECIMAL(5,2),
4      p_term_years INTEGER,
5      p_down_payment DECIMAL(10,2)
6  )
7  RETURNS DECIMAL(10,2)
8  AS $$
9  DECLARE
10     v_property_price INTEGER;
11     v_loan_amount DECIMAL(10,2);
12     v_monthly_rate DECIMAL(10,6);
13     v_num_payments INTEGER;
14     v_monthly_payment DECIMAL(10,2);
15 BEGIN
16     -- Получаем цену объекта
17     SELECT price INTO v_property_price
18     FROM property
19     WHERE id = p_property_id;
20
21     -- Проверяем, найден ли объект
22     IF v_property_price IS NULL THEN
23         RAISE EXCEPTION 'Объект недвижимости с id % не
24         найден', p_property_id;
25     END IF;
26
27     -- Рассчитываем сумму кредита
28     v_loan_amount := v_property_price - p_down_payment;
29
30     -- Проверяем корректность данных
31     IF v_loan_amount ≤ 0 THEN
32         RAISE EXCEPTION 'Первоначальный взнос превышает
33         стоимость объекта';
34     END IF;
35
36     -- Рассчитываем месячную процентную ставку
37     v_monthly_rate := (p_interest_rate / 100) / 12;
38
39     -- Количество платежей
40     v_num_payments := p_term_years * 12;
41
42     -- Рассчитываем ежемесячный платеж по формуле аннуитета
43     IF v_monthly_rate = 0 THEN
44         v_monthly_payment := v_loan_amount / v_num_payments;
45     ELSE
46         v_monthly_payment := v_loan_amount *
47             (v_monthly_rate * POWER(1 + v_monthly_rate,
48             v_num_payments)) /
49             (POWER(1 + v_monthly_rate, v_num_payments) - 1);
50     END IF;
51     RETURN ROUND(v_monthly_payment, 2);
52 END;
53 $$ LANGUAGE plpgsql;

```

```

real_estate_db=# \i 2.2/test/test_get_monthly_payment.sql
result
-----
38599.81
(1 row)

result
-----
19299.90
(1 row)

psql:2.2/test/test_get_monthly_payment.sql:8: ERROR:  Объект недвижимости с id <NULL> не найден
CONTEXT:  PL/pgSQL function calculate_monthly_payment_valekzhanin(integer,numeric,integer,numeric) line 16 at RAISE
result
-----
(1 row)

psql:2.2/test/test_get_monthly_payment.sql:14: ERROR:  Первоначальный взнос превышает стоимость объекта
CONTEXT:  PL/pgSQL function calculate_monthly_payment_valekzhanin(integer,numeric,integer,numeric) line 24 at RAISE

```

Задание 4

4. Формирует список объектов недвижимости, стоимость 1м² которых меньше средней по району

Вх. параметр	Вых. параметр
	адрес, название района, количество комнат

```

1  CREATE OR REPLACE FUNCTION
   get_properties_below_average_price_valekzhanin()
2  RETURNS TABLE(
3      address VARCHAR(255),
4      district_name VARCHAR(255),
5      room_count SMALLINT
6  )
7  AS $$
8  BEGIN
9      RETURN QUERY
10     SELECT
11         p.address,
12         d.name AS district_name,
13         p.room_count
14     FROM property p
15     JOIN district d ON p.district_id = d.id
16     WHERE (p.price::DECIMAL / p.area) < (
17         SELECT AVG(p2.price::DECIMAL / p2.area)
18         FROM property p2
19         WHERE p2.district_id = p.district_id
20         AND p2.area > 0
21     )
22     AND p.area > 0
23     ORDER BY d.name, p.address;
24 END;
25 $$ LANGUAGE plpgsql;

```

```

real_estate_db=# \i 2.2/test/test_get_properties_below_average_price.sql
address      | district_name | room_count
-----+-----+-----
ул. Водная, 6 | Западный      | 2
ул. Западная, 12 | Западный      | 2
ул. Каменная, 23 | Западный      | 2
ул. Речная, 7 | Западный      | 1
ул. Берёзовая, 9 | Северный      | 1
ул. Северная, 5 | Северный      | 1
ул. Снежная, 4 | Северный      | 2
ул. Старая, 2 | Северный      | 1
ул. Хвойная, 29 | Северный      | 1
пр. Победы, 17 | Центральный   | 4
ул. Береговая, 28 | Центральный   | 4
ул. Новая, 21 | Центральный   | 3
ул. Торговая, 26 | Центральный   | 3
ул. Зелёная, 16 | Южный         | 4
ул. Садовая, 8 | Южный         | 3
ул. Южная, 30 | Южный         | 4
(16 rows)

```

```

address      | district_name | room_count
-----+-----+-----
ул. Водная, 6 | Западный      | 2
ул. Западная, 12 | Западный      | 2
ул. Каменная, 23 | Западный      | 2
ул. Речная, 7 | Западный      | 1
ул. Берёзовая, 9 | Северный      | 1
ул. Северная, 5 | Северный      | 1
ул. Снежная, 4 | Северный      | 2
ул. Старая, 2 | Северный      | 1
ул. Хвойная, 29 | Северный      | 1
пр. Победы, 17 | Центральный   | 4
ул. Береговая, 28 | Центральный   | 4
ул. Новая, 21 | Центральный   | 3
ул. Торговая, 26 | Центральный   | 3
ул. Зелёная, 16 | Южный         | 4
ул. Садовая, 8 | Южный         | 3
ул. Южная, 30 | Южный         | 4
(16 rows)

```

Задание 5

5. Формирует статистику по продажам за указанный год. **Входные параметры:** год.

Тип объекта недвижимости	Количество продаж	Процент от общего количества все проданных ОН	Общая сумма
Квартир	5	16%	15000000
Домов	10	34%	18500000
Апартаменты	15	50%	20000000

```

1 CREATE OR REPLACE FUNCTION get_sales_statistics_valekzhanin
  (p_year INTEGER)
2 RETURNS TABLE(
3     property_type VARCHAR(255),
4     sales_count BIGINT,
5     percentage DECIMAL(5,2),
6     total_amount DECIMAL(15,2)
7 )
8 AS $$
9 DECLARE
10     v_total_sales BIGINT;
11 BEGIN
12     -- Получаем общее количество продаж за год
13     SELECT COUNT(*) INTO v_total_sales
14     FROM sale
15     WHERE EXTRACT(YEAR FROM sale_date) = p_year;
16
17     RETURN QUERY
18     SELECT
19         t.name AS property_type,
20         COUNT(s.id) AS sales_count,
21         CASE
22             WHEN v_total_sales > 0 THEN
23                 ROUND((COUNT(s.id)::DECIMAL / v_total_sales
24                     * 100), 2)
25             ELSE 0
26         END AS percentage,
27         COALESCE(SUM(s.price), 0) AS total_amount
28     FROM sale s
29     JOIN property p ON s.property_id = p.id
30     JOIN type t ON p.type_id = t.id
31     WHERE EXTRACT(YEAR FROM s.sale_date) = p_year
32     GROUP BY t.id, t.name
33     ORDER BY sales_count DESC;
34 END;
35 $$ LANGUAGE plpgsql;

```

```

CREATE FUNCTION
real_estate_db=# SELECT * FROM get_sales_statistics_valekzhanin(2023);
property_type | sales_count | percentage | total_amount
-----+-----+-----+-----
Квартира      |          14 |       70.00 | 84700000.00
Дом           |           4 |       20.00 | 40500000.00
Апартаменты   |           2 |       10.00 | 9800000.00
(3 rows)

```